CS151 Intro to Data Structures

Heaps II

Warmup

Insert the following keys one at a time into an initially empty min-heap:

15 3 8 1 12 6 9 4

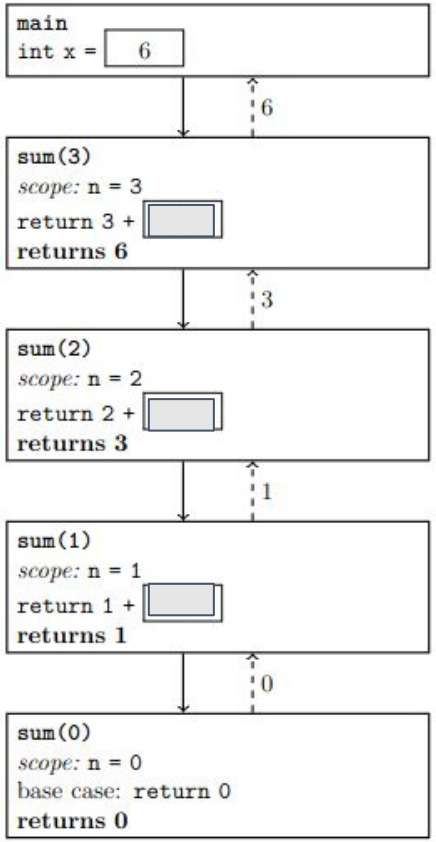
run the following operations:

- poll()
- insert(2)
- poll()

Recursion Review

```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum(n - 1);
}

// in main:
int x = sum(3);
```



solid arrow: the call
dashed arrow: the value returned to the caller

- Each time a method is called it gets its own box
- Each box holds all variables in the scope of that call
- solid vs dashed arrow

Warmup 2

Write a recursive method int `countLessThan(Node n, E x)` that returns how many elements in the subheap rooted at n are strictly less than x

Perform a runtime analysis

Announcements

HW5 due Friday

Lab7 due tonight

Ellen’s Office Hours (Thursday 5-7pm) moved to HC Hills Hall H110

CS Info Session on Thursday (4-5pm)

Outline

- Heap Review
- Array based binary trees

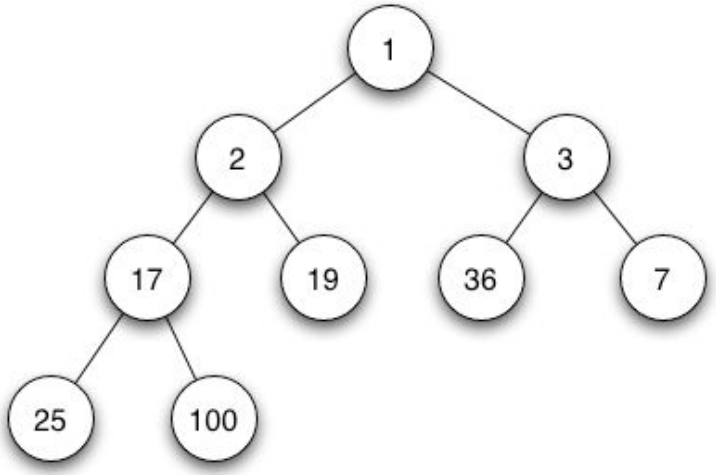
Heap Review

Binary Heap Properties

1. Each node has **at most 2 children**

1. For every node n (except for the root): **n.key >= parent(n).key**

1. **Complete:** all levels of the tree, except possibly the last one are fully filled, and, if the last level of the tree is not complete, the nodes of that level are filled from left to right



Heap Operations:

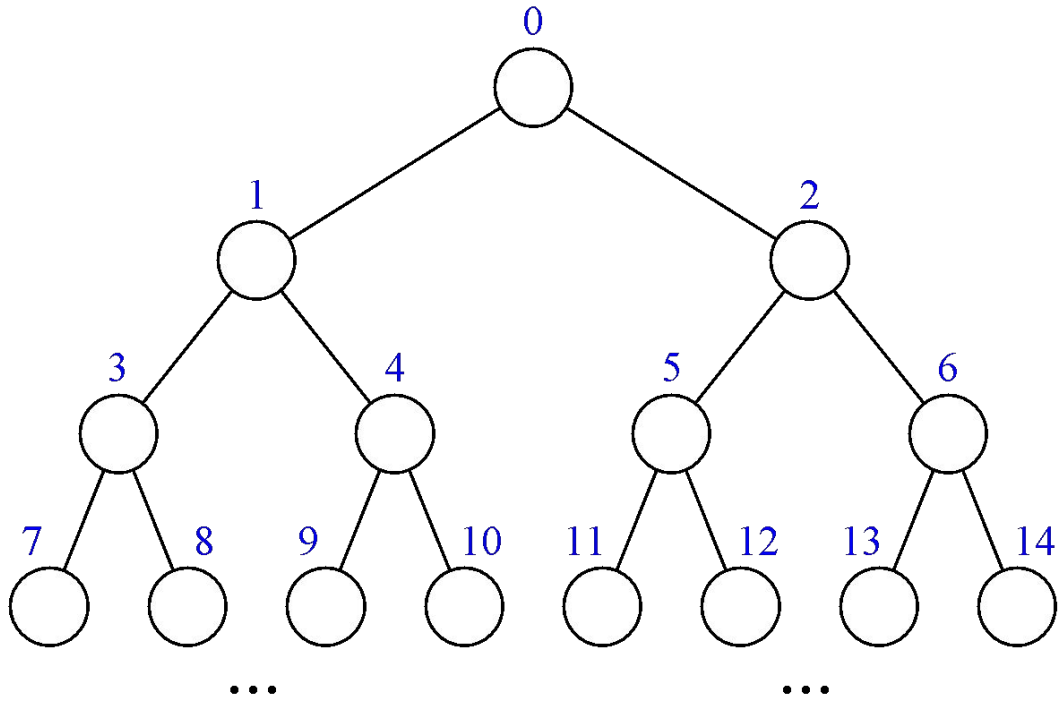
1. Insert
   a. Upheap
2. Poll
   a. Downheap
3. Search
4. Remove

Array-based Implementation

- BinaryTrees can be implemented in different ways
  - Linked nodes - what you're doing in lab
  - Array

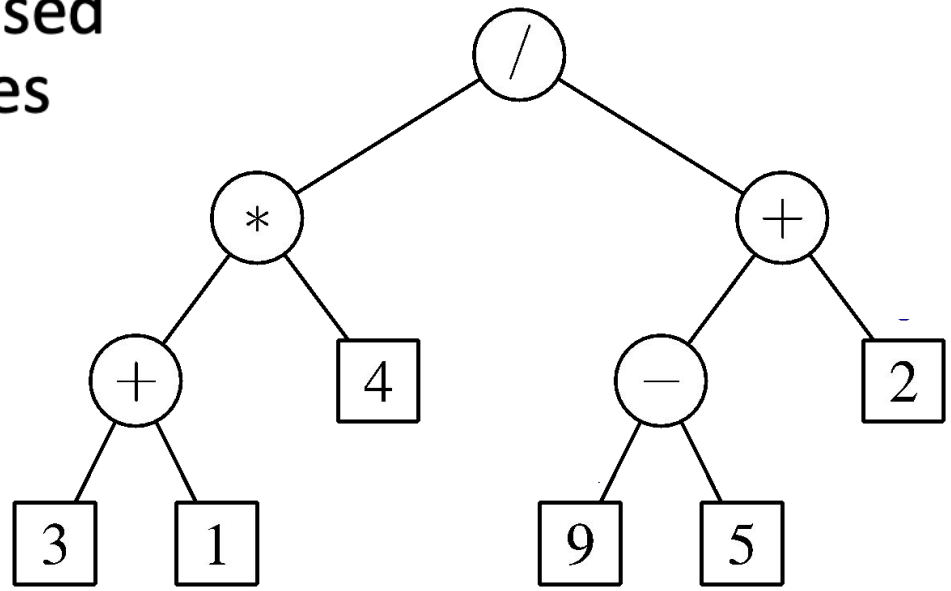
Array-based Implementation

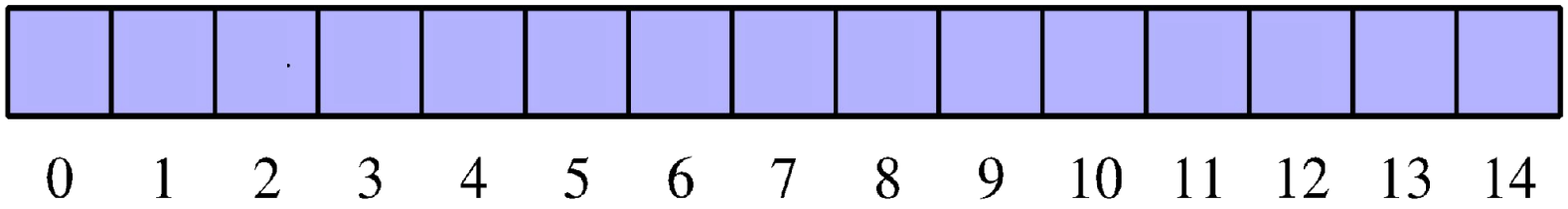
- Number nodes level-by-level, left-to-right
- $f(root) = 0$
- $f(l) = 2f(p) + 1$
- $f(r) = 2f(p) + 2$
- Numbering is based on all positions, not just occupied positions



Array-based Binary Tree

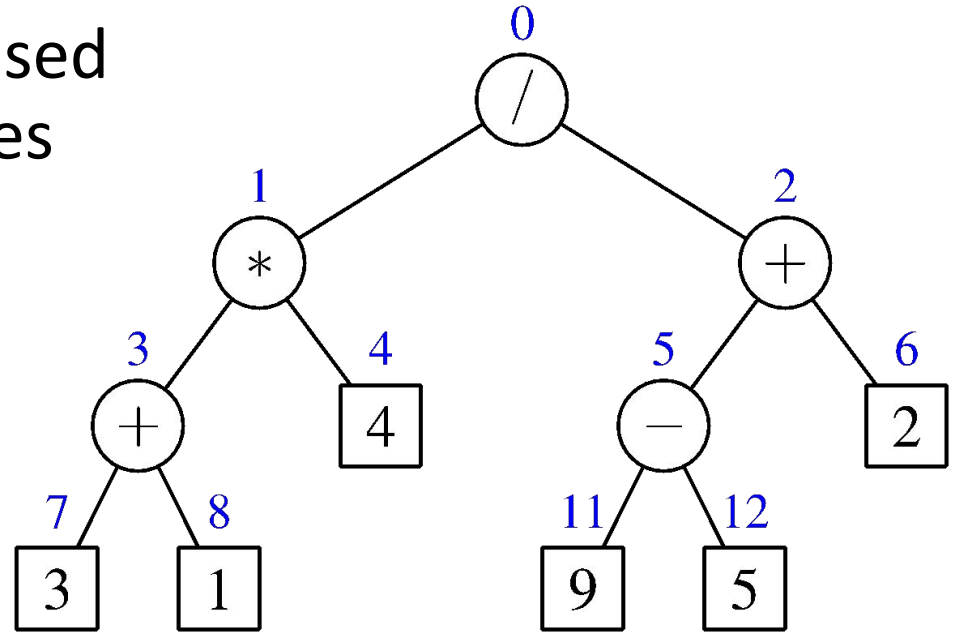
- The numbering can then be used as indices for storing the nodes directly in an array
- $f(root) = 0$
- $f(l) = 2f(p) + 1$
- $f(r) = 2f(p) + 2$

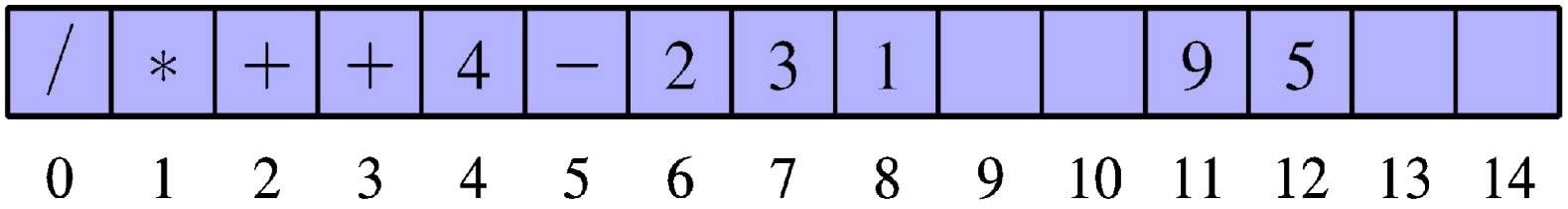




Array-based Binary Tree

- The numbering can then be used as indices for storing the nodes directly in an array



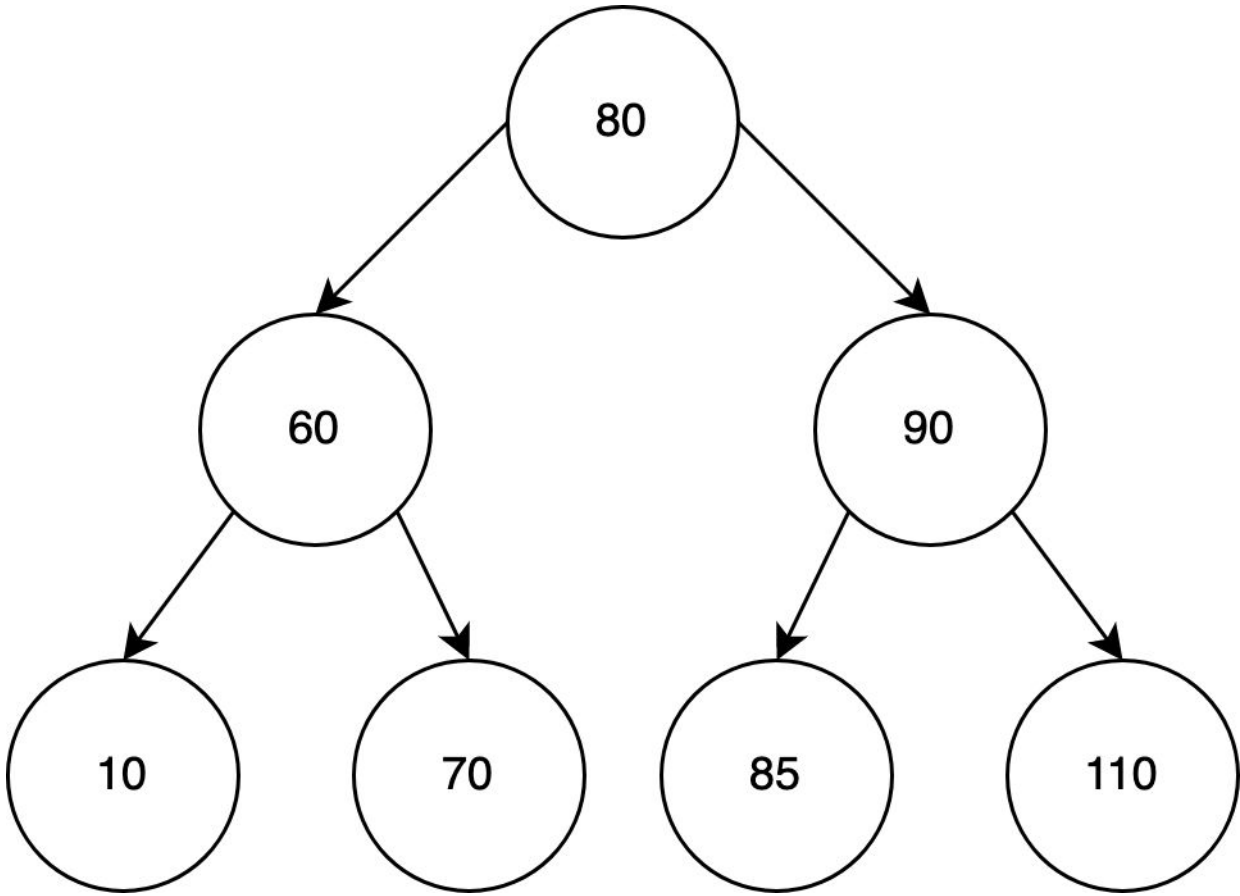


Array-based Binary Tree

What would the underlying array look like for this tree?

```
f(root) = 0
f(l)    = 2*f(p) + 1
f(r)    = 2*f(p) + 2
```

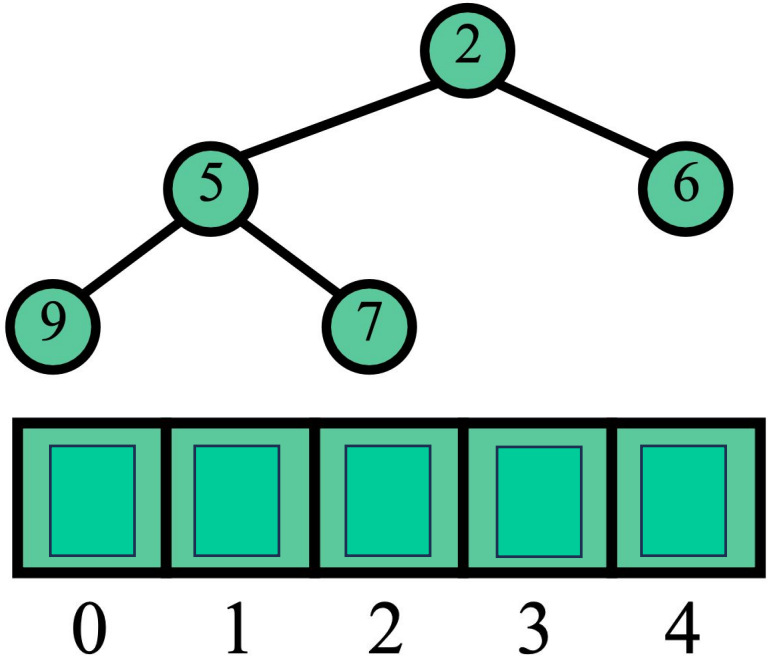
Now, let's insert 75. Where should it go?



Array based heap

Node at index i

- Left child index:
  - 2i + 1
- Right child index:
  - 2i + 2

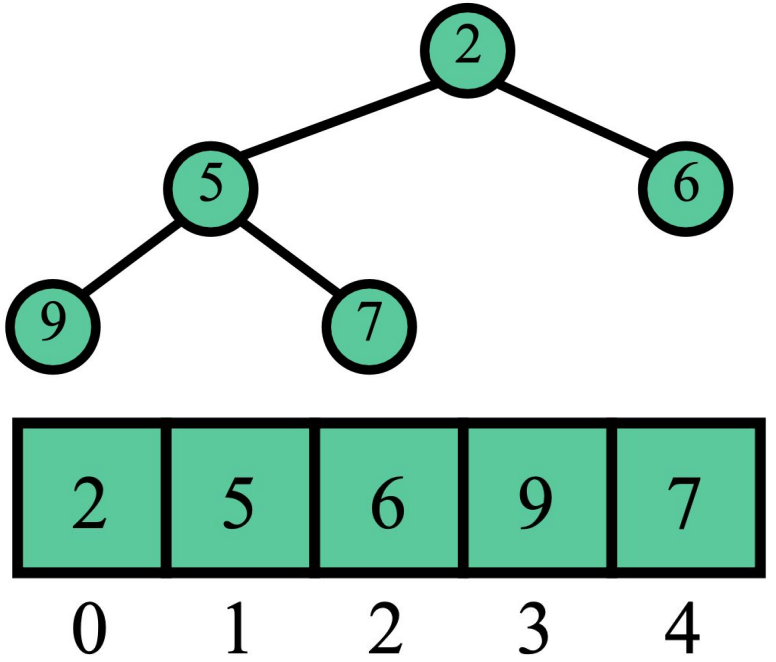


Array based heap

Node at index i

- Left child index:
  - 2i + 1
- Right child index:
  - 2i + 2

- Peek:
  - Get element at index 0
- Poll:
  - Remove element at index 0
- No need to store references/links



Repeat the warmup with an array heap

Insert the following keys one at a time into an initially empty min-heap:

15 3 8 1 12 6 9 4

run the following operations:

- poll()
- insert(2)
- poll()

Repeat warmup 2 with an array heap

Write a method int `countLessThan(E x)` that returns how many elements in the heap are strictly less than x

Perform a runtime analysis

Breadth First Search over Array Based Heap

Write a method `void printLevels(E[] heap, int size)` that prints the elements in breadth-first order, with each level of the tree on its own line. For example, the heap `[2, 5, 3, 9, 6, 4, 8, 12, 10, 7]` prints:

2

5 3

9 6 4 8

12 10 7

Summary

- Heaps
  a. DS for efficient removal of the min element

- Heaps can be represented as linked nodes or an array